

PROJECT REPORT

1. Project Title

Simple Currency Converter in Python

2. Executive Summary / Abstract

Simple Currency Converter: Command Line Python Program The Simple Currency Converter is a minimal command-line interface (CLI) program written in Python. It can transform amounts between five prominent global currencies, namely the Indian Rupee (INR), US Dollar (USD), Euro (EUR), Great Britain Pound (GBP), and Japanese Yen (JPY). The program defines INR as an anchor currency from which all the other values are derived, with thorough validation and a continuous CLI loop.

3. Introduction & Objectives

3.1 Background

Currency exchange – This is a one of the most frequently needed calculations for international commerce, finance, and travel. Command line-based applications are provide a speedy, efficient and inexpensive way of performing the calculations without GUI overheads.

3.2 Objectives

Dynamic Conversion: Effortlessly get an accurate conversion between any two available currencies using a base standard.

Easy to Use Navigation: Direct the user to request input interactively while also providing a list of the available currencies.

Better Error Handling: Handle gracefully non-numeric input and non supported currency codes without crashing.

Reusability & Modular: Write code in sensible functions to ease maintenance and extension.

4. System Requirements

4.1 Software Requirements

Operating System: Cross-platform (Windows, macOS, Linux)

Programming Language: Python 3.x

Libraries / Dependencies: Standard Python Library (sys, built-in types)

4.2 Hardware Requirements

Process: Intel Core i3 / AMD Ryzen 3 or similar

RAM: 512 MB minimum

Disk Space: Minimal (< 1 MB)

5. System Architecture & Methodology

5.1 Currency Conversion Logic

The program uses the Indian Rupee (INR) as the base rate (1.0). Convert from firstcurrency (fromcurrency) to secondcurrency (tocurrency) with the following mathematical formula:

Conversion to Base (INR):

$$\text{amount_in_inr} = \frac{\text{amount}}{\text{exchange_rates}[\text{from_currency}]}$$

Conversion from Base (INR) to Target Currency:

$$\text{converted_amount} = \text{amount_in_inr} \times \text{exchange_rates}[\text{to_currency}]$$

5.2 Exchange Rates Table

Currency Code

Currency Name

Exchange Rate (Relative to 1 INR)

INR

Indian Rupee

1.0

USD

US Dollar

0.012

EUR

Euro

0.011

GBP

British Pound

0.0095

JPY

Japanese Yen

1.78

6. Modules & Functions Description

`Exchange_rates` (Dictionary): Key-value store where key is a currency code (string) and value is their normal relative value against INR (float).

`Showcurrencies()`: Loops over the `exchangerates` dictionary and prints out all of the available currency codes to stdout.

`convertcurrency(amount, fromcurrency, to_currency)`:

Accepts amount and currency pairs.

Validates if both currencies exist in `exchange_rates`.

Returns None if an invalid currency was supplied. Otherwise returns `converted_amount`.

`main()`:

Serves as the driver program.

Runs an infinite loop of `while True` to take input from the user, run the logic, display the formatted output with two decimal places, and ask the user whether to continue or stop.

7. Test Cases & Execution Results

Test Case 1: Valid Conversion (USD to JPY)

Input:

FROM: USD

TO: JPY

Amount: 100

Output: 100.0 USD = 14833.33 JPY

Test Case 2: Invalid Amount Input

Input:

FROM: INR

TO: USD

Amount: abc

Output: Invalid amount. Please try again. (Insures user enters a valid number)

Test Case 3: Unsupported Currency Code

Input:

FROM: CAD

TO: USD

Output: Invalid currency code. Please select from the list of supported currencies.

8. Key Features & Strengths

Input normalization: `with.strip().upper()` and `strip().lower()` makes extra whitespace and case insensitivity work out of the box.

Formatting: Formats the returned floating point value to 2 decimal places (`:.2f`).

Solid Error Handling: Try-except blocks for type testing.

9. Future Enhancements & Scope

Real-Time API Integration: Retrieve real-time market exchange rates dynamically using an API (exchangerate-api, fixer.io).

Graphical User Interface (GUI): Create a user interface with tkinter or PyQt.

Database Integration: Store the history of conversions in SQLite or PostgreSQL to keep a record of former conversions.

Support Additional Currencies Expand the currency dictionary to cover all worldwide currencies and cryptocurrencies.

10. Conclusion

Simple Currency Converter shows how the core programming skills of the Python language can be successfully combined. The project illustrates: using dictionary lookups in Python functions breaking a program into functions handling input validation looping through a set of options.